Design of Analog-AI Hardware Accelerators for Transformer-based Language Models (Invited)

G. W. Burr[1], H. Tsai[1], W. Simon[2], I. Boybat[2], S. Ambrogio[1], C.-E. Ho[1], Z.-W. Liou[1], M. Rasch[3], J. Büchel[2], P. Narayanan[1], T. Gordon[4], S. Jain[3], T. M. Levin[4], K. Hosokawa[5], M. Le Gallo[2], H. Smith[1], M. Ishii[5], Y. Kohda[5], A. Chen[1], C. Mackin[1], A. Fasoli[1], K. El Maghraoui[3], R. Muralidhar[3], A. Okazaki[5], C.-T. Chen[3], M. M. Frank[3], C. Lammie[2], A. Vasilopoulos[2], A. M. Friz[1], J. Luquin[1], S. Teehan[4], I. Ahsan[4], A. Sebastian[2], and V. Narayanan[3]

[1]IBM Research–Almaden, San Jose, CA, USA email: *gwburr@us.ibm.com*
[2]IBM Research–Zurich, Rüschlikon, Switzerland [3]IBM Research–T. J. Watson Research Center, Yorktown Heights, NY, USA
[4]IBM Albany NanoTech, Albany, NY, USA [5]IBM Tokyo Research Laboratory, Tokyo, Japan.

2023 International Electron Devices Meeting (IEDM) | 979-8-3503-2767-0/23/$31.00 ©2023 IEEE | DOI: 10.1109/IEDM45741.2023.10413767

***Abstract*—Analog Non-Volatile Memory-based accelerators offer high-throughput and energy-efficient Multiply-Accumulate operations for the large Fully-Connected layers that dominate Transformer-based Large Language Models. We describe architectural, wafer-scale testing, chip-demo, and hardware-aware training efforts towards such accelerators, and quantify the unique raw-throughput and latency benefits of Fully- (rather than Partially-) Weight-Stationary systems.**

***Index Terms*—In-memory computing, Non-volatile memory, large language models, analog multiply-accumulate for DNN inference, analog AI, deep learning accelerator, system modeling**

I. Introduction

Scaling the *size* of Deep Neural Network (DNN) models has been a key driver of progress in artificial intelligence, first in image recognition [1] and more recently in Large Language Models (LLMs) [2]. This trend has greatly increased the amount of compute and energy needed to train these very large networks with vast datasets [3, 2], as well as to perform rapid inference on the resulting highly capable models [2]. The Graphics Processing Units (GPUs) used for these compute tasks have scaled greatly in capability (and quantity), but only modestly in energy-efficiency. Reductions in computational precision since 2015 [4] have now mostly saturated, and the benefits of sparsity [5] are much less applicable to the large Fully-Connected (FC) layers found in LLMs – leaving technology node as the primary path to still higher efficiency.

Recently, numerous researchers are exploring Compute-In-Memory (CIM) approaches [6, 7, 8] to increase energy-efficiency by performing Multiply-Accumulate (MAC) operations within ON-chip memory "Tiles," thus markedly reducing the motion of model-weights and partial sums. By exploiting such weight stationarity over short timespans with Volatile Memories (VMs) – SRAM or e-DRAM [6, 8] – or over longer timespans with slower, finite-endurance Non-Volatile Memories (NVMs) – Flash [9], Resistive-RAM [10], or phase-change memory (PCM) [11, 12] – such CIM approaches can offer high throughput, low latency, and high energy-efficiency [6, 7, 8].

II. Transformer-based networks

While these benefits can be seen across all DNN models, they are particularly pronounced for workloads that exhibit large FC layers, such as **Transformer-based models** [14, 13, 2]. Such models, including BERT-base [13], BERT-large [13], and GPT-3 [2], use a stack of 12, 24, or 96 identically-structured layers, each comprising 4 large FC-layers (In-projection, Out-projection, FC1, FC2) and a self-attention block (**Fig. 1**). FC-compute operations (OPs) typically dominate the workload, but scale linearly with sequence-length S; attention-compute OPs scale as S^2.

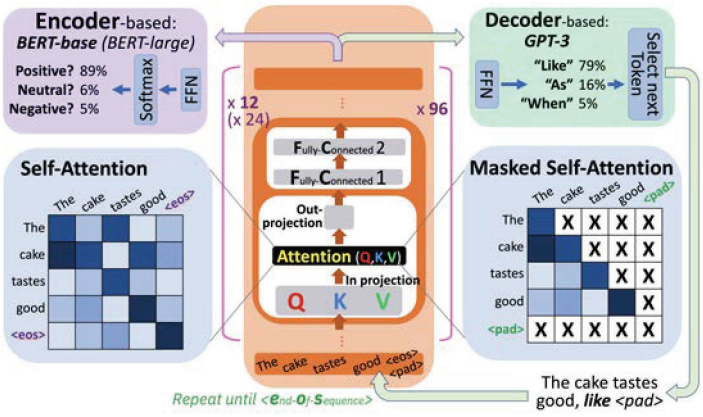


Fig. 1: Transformer-based models push sequences of tokens (vectors encoding words) through a stack of layers, each layer comprising 4 large FC-blocks and a self-attention (SA) block. Decoder-based architectures use a masked SA-block.

Encoder-based models perform Natural Language Processing (NLP) classification-tasks (sentiment analysis, etc.) by pushing an entire sequence of S "tokens" through the layer-stack, using all S^2 elements of each attention-matrix (within each of the attention-heads, within each layer, **Table 1**). *Decoder-based* models generate one or more new tokens with a similar pass, except the attention-block uses masking and sparsity to reduce the amount of attention compute [2, 15] to support long sequences. In an encoder-stack, tokens need to be staged before each attention-compute block, requiring significant local scratchpad and disrupting pipelining. In contrast, the *masked self-attention* in a decoder-stack can proceed in a more pipelined manner, needing only a few incoming tokens before outgoing tokens can be injected into the subsequent Out-Projection block. Note that encoder-based models are trained in two steps – extensive *pre-training* that typically only needs to be performed once, followed by *fine-tuning* for the particular classification task at hand. Ultra-large LLMs are so capable after pre-training that sometimes only simple "prompt engineering" is required.

The sheer size of these Transformer-based LLMs calls for more efficient AI compute. While the *macro*-level energy-efficiency of a CIM-Tile is demonstrably high [6, 8], it is non-trivial to deliver high *system* energy-efficiency — but careful design can implement the necessary Auxiliary digital compute, and can communicate excitation-vectors between Tiles and Compute-cores with reasonable energy and time [16]. VM-based CIM systems also need to occasionally move weight-vectors, in order to reuse the same small set of Tiles for many different FC-layers comprising different weight matrices.

III. Analog-Fabric Architecture

We recently introduced a highly programmable Analog-AI and Compute-In-Memory accelerator architecture [16], which utilizes a highly specialized set of Compute-cores and Tiles arranged within a common building block known as an Analog Fabric (AF) (**Fig. 2**), together with SRAM scratchpads, IO blocks, and a 2D Mesh on top of the building blocks. Border-Guard circuits (BGs) are placed between components to control dataflow over the 2D mesh for high throughput and energy-efficiency across a wide variety of workloads, supporting multi-casting of data-vectors traveling from Compute-cores to Tiles. This on-chip interconnect helps enable system-level utilization and efficiency well beyond current AI accelerator systems. Multiple AFs can share the same chiplet, which can be tiled for system scale-up.

	BERT-base [13]	BERT-large [13]	GPT-3 [2]
type	encoder	encoder	decoder
# of layers	12	24	96
hidden vector-size	768	1024	12,288
# of attn. heads/layer	12	16	96
head-size	64	64	128
sequence-length	**128** – 512	**128** – 512	2,048
FC : attn. compute	97% : 3%	98% : 2%	$\gg$95% : $\ll$5%
# of weights	85M	302M	174B
Tile-size assumed	512	512	1024
tiles/layer	38	48	1,728
CC_H cores/layer	3	4	24

Table 1 – Transformer-based DNN models

Authorized licensed use limited to: Princeton University. Downloaded on October 12,2026 at 21:42:06 UTC from IEEE Xplore. Restrictions apply.

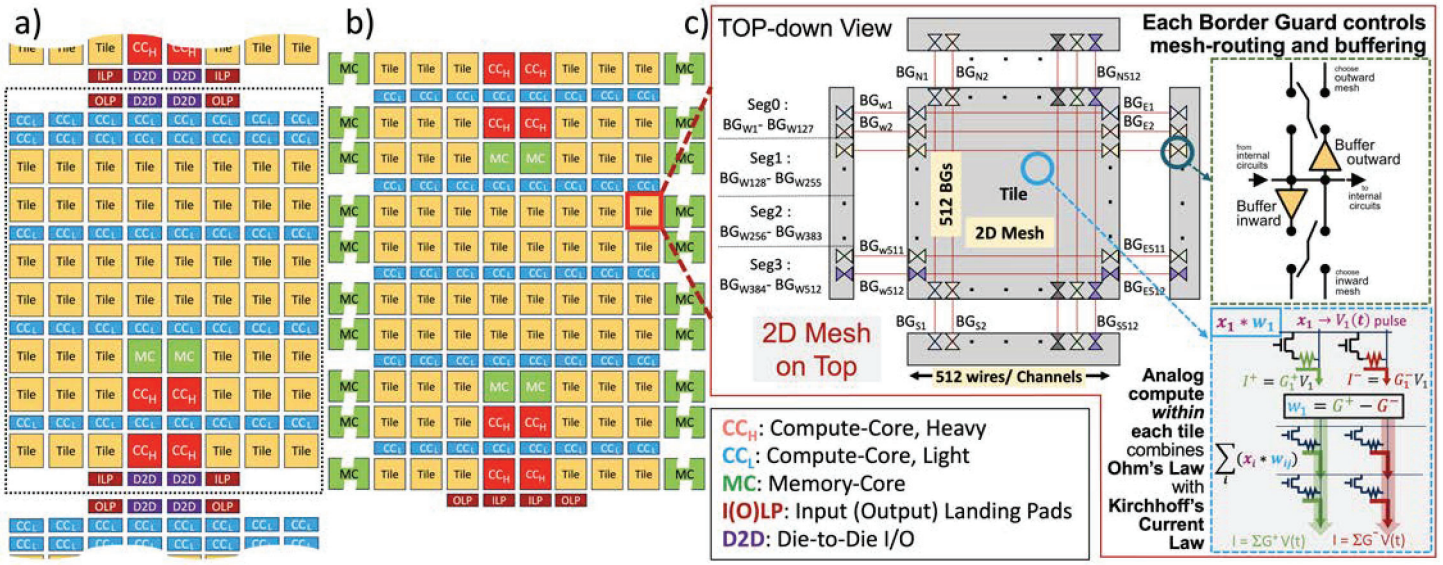


Fig. 2: A base chiplet is either a) repeated vertically to create a Fully-Weight-Stationary (FWS) system, or b) reused by weight-reloading to create a Partially-Weight-Stationary (PWS) system. Blocks such as analog-memory Tiles, Heavy and Light Compute-cores (CC_H and CC_L), scratchpads (MC), and both off-chip and Die-to-Die (D2D) IO are interconnected using c) a 2D Mesh with BorderGuard circuits (BGs) for dataflow control. In an FWS system, both Ohm's law and Kirchhoff's law allow Analog-AI compute using Non-Volatile Memory (NVM)-based resistors; in a PWS system, weight-reloading mandates high-endurance SRAM (or e-DRAM), using Kirchhoff's law or charge sharing for MAC computation with multiple word- and/or bit-lines (not shown) [8], and large Memory-Cores (or off-chip DRAM, not shown) to store layer-weights. (Part (c) adapted from [16].)

The Tiles are the primary MAC-compute and weight storage units for the DNN inference workload. In NVM-based accelerators, the conductance G of the analog resistive device implements synaptic weight, while voltage level or duration implements input activations. Activations arrive as parallel bit-streams via the 2D Mesh and are converted into analog pulse durations, enabling parallel excitation of the analog memory array. Multiply-accumulation is performed by the combination of Ohm's and Kirchhoff's Current Laws (**Fig. 2c**) [17, 7]. The resulting raw summations are converted from analog current and integrated charge back into digital vectors, which are then transmitted using the 2D Mesh. Conversion is performed in parallel using dedicated Current-Controlled Oscillator (CCO)-based Analog-to-Digital Converters (ADCs) [12].

Given the finite endurance, and the slow and power-hungry programming of NVM-devices, such Analog-AI systems must be Fully-Weight-Stationary (FWS) – every weight must be pre-programmed before execution of the inference workload begins. While this requires unique NVM-devices for each and every weight in the model, there are numerous benefits. No time or energy is spent moving weights during the workload, allowing the maximum degree of pipelining supported by the model. Adverse tradeoffs between efficiency and latency are avoided, so that the system inherently reaches maximum throughput *and* energy-efficiency at batch-sizes as low as b=2. Excess padding of short sequences is also not required.

By optimizing a spatial distribution of Tiles and Compute-cores to meet the needs of diverse networks (including LSTMs, Transformers, and CNNs), together with a well-optimized 2D mesh-interconnect, we showed that this Analog-AI CIM-based architecture can be 40×-140× more energy-efficient than the NVIDIA A100 (compared at b=8) across networks of interest [16]. The NVM-Tile assumed here is based on PCM, with characteristics (100 TOPS/W at 40 nsec integration-time) taken from 14nm-node circuit-simulations [16].

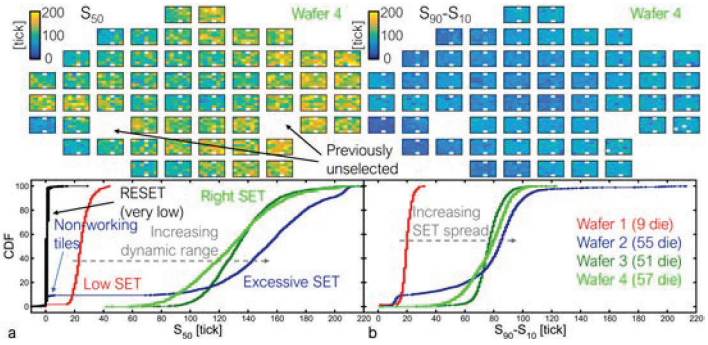


Fig.3: Full Wafer Maps and CDFs for the (a) S_{50} and (b) S_{90}-S_{10} spread, indicating the conductance change, expressed in digital integration counts, for the median (50%) SET-state conductance value and the spread between the 10% and 90% SET conductance.

We now turn to various challenges involving devices (accurate programming of NVM devices within high-yield arrays), array- and chip-circuitry (reducing nonlinearities, orchestrating vectorized communication, and efficiently performing specialized digital-compute), and algorithms (preparing model weights, maximizing DNN accuracy).

IV. Wafer-scale testing

The vast number of NVM CIM-Tiles needed (**Table 1**) must be implemented at high-yield. While identifying completely dead chips with *Wafer-Scale Testing* (WST) is straightforward, flagging PCM-based Analog-AI chips that "work, but not very well" is more difficult. Unlike a digital memory characterized by bit-error-rate and fast write-latency, an Analog-AI array requires precise conductance-tuning across broad resistance range. While array readout must be fast (nanosecond regime), models are installed infrequently — so slow and complex iterative-write-and-verify schemes that can maximize DNN accuracy by reducing write-programming errors are preferred. Yet such iterative schemes are far too slow for screening entire wafers. So we need to gauge the suitability of the PCM arrays for Analog-AI using a very small number of conductance-reads per device.

Since device-to-device variability primarily manifests as broadened RESET (lowest-conductance) and SET (highest-conductance) distributions, we can perform efficient WST for multicore PCM-based Analog AI by examining the sharpness of the RESET and SET Cumulative Distribution Functions (CDFs). Devices in an ideal array should exhibit nearly-identical RESET and SET resistances with large contrast ($> 5\times$) [20]. **Fig. 3** shows how this maximum achievable SET conductance varies across Tiles, chips, and a full wafer. Chips identified by WST as having either poor contrast or excessively high SET distributions tend to exhibit poor weight-programming accuracy.

V. Chip demonstrations of various DNN

While chiplets with high NVM-yield are necessary for accurate DNN inference, they are not sufficient. Each NVM CIM-Tile must implement accurate MAC operations, despite the presence of device and circuit noise, circuit nonlinearities, and quantization effects. The resulting excitation vectors must then efficiently move between Tiles and Compute-cores used for efficient Auxiliary compute, leading to DNN results at high throughput, high energy-efficiency, and low-latency – delivered at- or near the software-equivalent DNN accuracy

Authorized licensed use limited to: Princeton University. Downloaded on October 12,2026 at 21:42:06 UTC from IEEE Xplore. Restrictions apply.

			Test Error in %						Normalized acc. in %			
Source	DNN	model-size	FP32	1 sec	1 hour	1 day	1 week	1 year	$\mathcal{A}_*^{1h}$	$\mathcal{A}_*^{1d}$	$\mathcal{A}_*^{1w}$	$\mathcal{A}_*^{1y}$
HWA [18]	RNN-T SWB300	57M	11.8	12.22	12.36	12.42		12.78	**99.4**	**99.3**		98.9
chip [19]	RNN-T LibriSpeech	45M	7.452	9.258			9.894		98.0		97.4	
HWA [18]	BERT-base GLUE8	85M	17.47	17.43	17.55	17.58		17.99	**99.8**	**99.8**		98.9
HWA [18]	ALBERT-base GLUE8	7.8M	19.46	20.52	20.45	21.08		22.18	97.8	96.4		94.0
HWA [this work]	RoBERTa-base GLUE8	85M	15.83	16.67	16.76	16.96	16.97	17.43	98.1	97.7	97.7	96.7

Table 2 – DNN accuracy from Hardware-Aware Training [18] and chip-testing [19], plus new results for HWA training of RoBERTa-base.

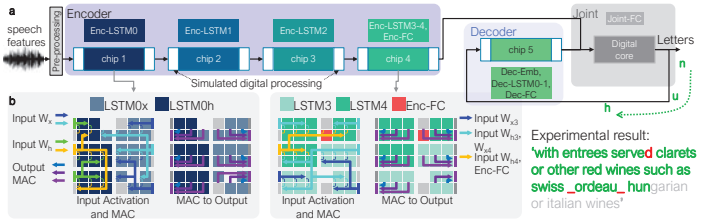


Fig.4: (a) RNNT model transcribes input speech into words, character by character, using compute blocks mapped onto 5 chips (plus off-chip digital-compute). 45M weights are implemented with ∼140M PCM devices, (b) with complex routing and multicasting of input- and MAC-vectors across 34 Tiles. Example sentence from experimental results, at 9.258% Word-Error-Rate [19].

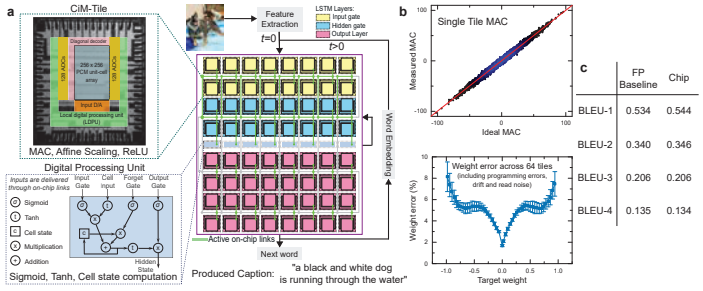


Fig. 5: a) A 64-Tile chip combines CCO-based ADCs and local digital-compute (scaling, ReLU activations) at each Tile, together with more complex digital processing (sigmoid & tanh activations, LSTM-vector OPs) at chip-center, to implement an image-captioning LSTM model. b) Highly-linear MAC operations and accurate weight-programming resulted in c) software-equivalent BLEU scores.

exhibited by the original DNN model (as implemented on a CPU or GPU using 32-bit Floating-Point (FP32) compute). We describe several chip-demonstrations that extend the boundaries of total number of DNN weights implemented and complexity of excitation orchestration, the integration of highly-linear, well-characterized and optimized NVM CIM-Tile compute, and the implementation of Auxiliary compute at or near the CIM-Tiles.

Recently, a large Recurrent Neural Network Tranducer (RNNT) model (**Fig. 4a**) was demonstrated [19], using 5 chips to encode 45M weights using >140M PCM devices. Tightly-orchestrated transport of massively-parallel excitation vectors was implemented using a 2-D mesh similar to **Fig. 2** (here conveying durations rather than digital-bits). While nearly all MAC operations are performed on-chip, the remaining digital operations are performed off-chip. Near software-equivalent accuracy (<2% degradation in Word Error Rate) was measured on the Librispeech dataset, along with an estimated 14× benefit in system performance, expressed as TOPS/W or Samples/s/W, over existing MLPerf digital submissions based on GPUs. This chip-demo implements a large number of DNN weights, together with experimental demonstration of the 2-D circuit-switched mesh concept.

On a second chip, we integrated CCO-based ADCs (one per integration-row) into PCM-based CIM-Tiles [21] (**Fig. 5**). This chip also features digital Auxiliary-compute for affine-scaling and complex activation-functions (ReLU, sigmoid, tanh) at or near the CIM-Tiles, and was used to demonstrate an LSTM-based image-captioning network with high accuracy (**Fig. 5**). All computation (fully-connected layers, LSTM vector-operations) were performed on-chip; excitation-vectors were pulled off-chip for data-motion between chip-blocks.

VI. Hardware-Aware Training of DNNs

We can further increase the robustness of DNN models by performing hardware-aware (HWA) training, preparing the network for the various imperfections in the Matrix-Vector-Multiplications (MVMs) performed on NVM-based CIM-Tiles. These arise from a variety of noise sources, nonlinearities and quantization effects. HWA training requires a model that is accurate enough to be predictive while still being amenable to the rapid computation needed to perform DNN training in reasonable time. HWA is analogous to quantization-aware training (QAT) methods now commonly used when deploying DNNs onto reduced-precision digital hardware [22].

For instance, the image captioning network shown in **Fig. 5** was trained with HWA techniques using a noise-model carefully matched to measured characteristics of the NVM-CIM tiles on the 64-Tile chip. In fact, starting with a model trained to high-accuracy with FP32, HWA training can greatly enhance the robustness of a variety of deep neural networks (DNNs) including recurrent neural networks (RNNs), and transformers (**Table 2**) as well as convolutional neural networks (CNNs) (not shown here, see Ref [18, 21]). In general, recurrent networks (such as LSTMs and RNNT) are the most robust to NVM-based CIM-Tile imperfections. Transformer models are reasonably robust, although models with fewer degrees-of-freedom tend to be less robust. For instance, ALBERT has the same number of operations as BERT-base but reuses the the same weights for all layers. While this may decrease the model footprint, there is a cost in both raw test-error performance and in robustness. In contrast, using HWA training techniques to perform the fine-tuning for each GLUE classification-task can provide fully software-equivalent accuracy on BERT-base, at least until the accumulated effects of one year of PCM conductance-drift after programming [18]. HWA training of RoBERTa – an optimized version of BERT trained on a much larger dataset – can provide even lower absolute test-errors (**Table 2**).

As a result, we should be able to prepare many high-yield chiplets organized into an NVM-based Analog Fabric, accurately program model weights made highly robust with HWA-based fine-tuning, and have an expectation to obtain the same accuracies as any digital system using ∼4-bit weights and 8-bit excitations, with high energy-efficiency.

VII. Benefits of Full Weight Stationarity

But one might ask: "Why make the effort to provide all the NVM CIM-Tiles needed for a Fully-Weight-Stationarity (FWS) system? Since a Partially-Weight-Stationary (PWS) system can greatly reduce the number of Tiles, perhaps weight-reloading can be infrequent enough that it is not troublesome?" To answer these questions, we employ the same performance simulator used in Ref [16].

The AF used in Ref [16] was optimized for performance across a wide variety of workloads. Here we introduce two new AFs **Fig. 2(a,b)**, designed specifically for BERT-base and BERT-large, containing exactly enough tiles, MC-cores, CC_L and CC_H cores for one layer of BERT-large (a few Compute-cores and Tiles are power-gated for BERT-base). Die-to-Die (D2D) I/O cores are added to provide sufficient interconnect between FWS chiplets. Both systems use exactly the same Tile characteristics (100 TOPS/W, 40nsec integration). This Tile performance (10.9 TOPs/sq.mm) matches or exceeds existing SRAM-based CIM-Tiles [23].

Our AF architecture was adapted for VM-based CIM tiles, to implement a highly-efficient PWS system by adding additional scratchpad (MC) cores (**Fig. 2(b)**), both for staging of excitations ($b \times S$ tokens) each time the workload passes through the currently-programmed VM-CIM tiles, and for weight-storage across all the layers (up to 12 MB, 10 sq.mm per MC). (Alternatively, weights could be loaded from DRAM, albeit at a significant energy cost.) To reduce data-traffic and reduce weight rewriting, odd layers progress from south to north, even layers from north to south. These choices, and the fraction of the network implemented between reloading, will drive interesting

Authorized licensed use limited to: Princeton University. Downloaded on October 12,2026 at 21:42:06 UTC from IEEE Xplore. Restrictions apply.

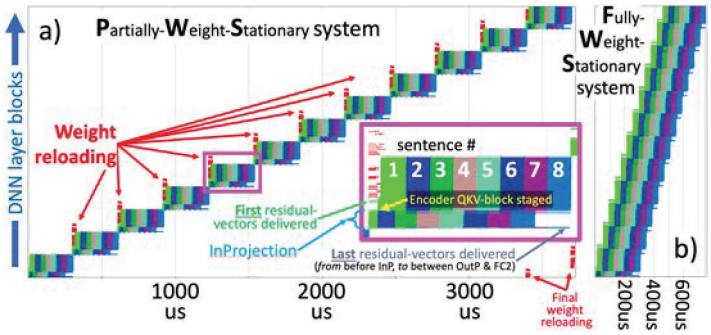


Fig.6: Progress through BERT-base DNN-layer-blocks vs. time for the (a) PWS and (b) FWS systems. The timing of the 12-layers and b=8 sentences, the staging of all sentence-tokens before initiating self-attention (QKV) compute, and the staging of residually-connected data are all visible. PWS runs include weight-reloading before a CIM-Tile can be reused with different weights, and after the workload (to return the system to its initial state).

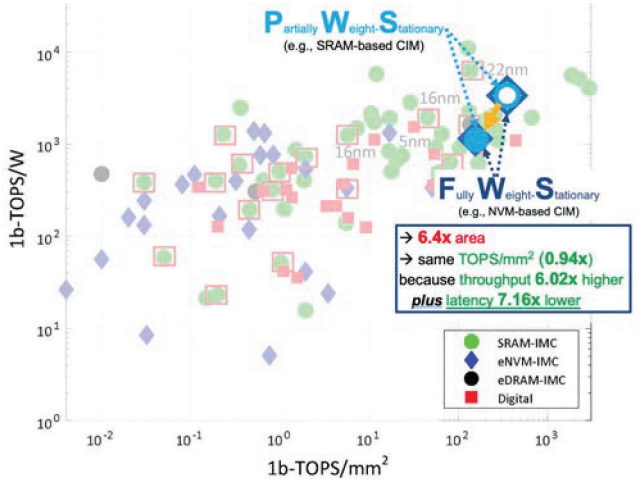


Fig. 7: 1-bit TOPS/W vs. 1-bit TOPS/sq.mm examples [8], with new data points from this study marked for Partially-Weight-Stationary (cyan solid circle) and Fully-Weight-Stationary (blue solid diamond) solutions capable of implementing BERT-large. Identical CIM-Tile performance (100 TOPS/W, 40nsec raw Tile-compute, open symbols) are assumed for PWS (e.g., SRAM-based CIM) and FWS (e.g., PCM-based CIM [16]), assuming 8-bit excitations and 4-bit (equivalent) weights.

tradeoffs across sequence-length and batch-size (here S=128 and b=8), which we plan to study carefully in future work.

Fig. 6 shows the trajectory of the 8 sentences through the 12 layers of BERT-base, for both PWS and FWS systems, and **Table 3** shows the results. Although the FWS system does occupy more area, the system throughput is significantly higher. Thus TOPS/sq.mm is maintained while offering markedly reduced latency — a benefit that is difficult to recover by simply duplicating the PWS system out to iso-area. **Fig. 7** puts these quantitative performance results in context to each other and to prior SRAM- and NVM-based CIM studies.

12-layer BERT-base	Partially WS	Fully WS	FWS is...
Area [sq.mm]	210	12×94	**5.3**× larger, but...
TOP/sec	47.5	235	**4.95**× faster
TOP/sec/W	27.4	29.1	**1.06**× as energy-efficient
TOP/sec/sq.mm	0.23	0.21	**0.93**× as area-efficient, but
latency [ms]	3.70	0.55	**6.75**× faster
24-layer BERT-large	**Partially WS**	**Fully WS**	**FWS is...**
Area [sq.mm]	350	24×94	**6.4**× larger, but...
TOP/sec	82.2	495	**6.02**× faster
TOP/sec/W	33.6	35.5	**1.06**× as energy-efficient
TOP/sec/sq.mm	0.23	0.22	**0.94**× as area-efficient, but
latency [ms]	7.58	1.06	**7.16**× faster

Table 3 – Performance modeling results: Fully- vs. Partially-Weight-Stationary

VIII. Conclusion

Analog non-volatile memory (NVM)-based accelerators can offer high-throughput and energy-efficient multiply-accumulate (MAC) operations for the large Fully-Connected (FC) layers that dominate Transformer-based Large Language Models (LLMs). We describe a highly heterogeneous and programmable accelerator architecture for DNN inference that combines Compute-In-Memory (CIM) Tiles for weight-stationary, energy-efficient MAC operations, together with heterogeneous special-function Compute-cores for auxiliary digital computation. Massively parallel vectors of neuron-activation data are exchanged over short distances using a dense and efficient circuit-switched 2D mesh, with analog compute implemented using Ohm's law and Kirchhoff's current law on arrays of resistive memory devices. If weights are accurately programmed onto NVM devices and MAC operations are sufficiently linear, competitive DNN accuracies can be obtained, aided by Hardware-Aware training techniques. Even though a Fully-Weight-Stationary system does occupy more area, it offers so much throughput that it effectively pays for itself in terms of TOPS/sq.mm. We show that an FWS system can offer a 6× higher raw-throughput and 7× faster latency.

IX. Acknowledgement

We thank W. Wilcke, S. Narayan, S. Munetoh, S. Yamamichi, C. Osborn, J. Burns, R. Divakaruni, M. Khare and the IBM Research AI Hardware Center for management and logistical support.

References

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton. "Imagenet classification with deep convolutional neural networks". In: *Adv. Neur. Inf. Proc. Sys.* 25 (2012).
[2] T. B. Brown et al. "Language Models are Few-Shot Learners". In: *arXiv:2005.14165* (2020).
[3] D. Amodei and D. Hernandez. *AI and Compute*. URL: https://openai.com/blog/ai-and-compute/.
[4] S. Gupta et al. "Deep Learning with Limited Numerical Precision". In: *ICML*. June 2015, pp. 1737–1746.
[5] T. Hoefler et al. "Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks". In: *J. Mach. Learn. Res.* 22.1 (2021), pp. 10882–11005.
[6] N. Verma et al. "In-Memory Computing: Advances and Prospects". In: *IEEE Sol.-State Circ. Mag.* 11.3 (2019), pp. 43–55.
[7] G. W. Burr et al. "Ohm's Law + Kirchhoff's Current Law = Better AI: Neural-Network Processing Done in Memory with Analog Circuits will Save Energy". In: *IEEE Spectrum* 58.12 (2021), pp. 44–49.
[8] N. R. Shanbhag and S. K. Roy. "Benchmarking In-Memory Computing Architectures". In: *IEEE Open J. Sol.-State Circ. Soc.* 2 (2022), pp. 288–300.
[9] L. Fick et al. "Analog matrix processor for edge AI real-time video analytics". In: *ISSCC*. Vol. 65. 2022, pp. 260–262.
[10] J.-H. Yoon et al. "A 40nm 100Kb 118.44 TOPS/W Ternary-weight Compute-in-Memory RRAM Macro with Voltage-sensing Read and Write Verification for reliable multi-bit RRAM operation". In: *CICC*. 2021, pp. 1–2.
[11] P. Narayanan et al. "Fully On-Chip MAC at 14 nm Enabled by Accurate Row-Wise Programming of PCM-Based Weights and Parallel Vector-Transport in Duration-Format". In: *IEEE Trans. Electr. Dev.* 68.12 (2021), pp. 6629–6636.
[12] R. Khaddam-Aljameh et al. "HERMES-core—A 1.59-TOPS/mm 2 PCM on 14-nm CMOS in-memory compute core using 300-ps/LSB linearized CCO-based ADCs". In: *IEEE J. Solid State Circ.* 57.4 (2022), pp. 1027–1038.
[13] J. Devlin et al. "BERT: Pre-training of deep bidirectional transformers for language understanding". In: *arXiv preprint arXiv:1810.04805* (2018).
[14] A. Vaswani et al. "Attention is All you Need". In: *Adv. Neur. Inf. Proc. Sys.* 2017, pp. 5998–6008.
[15] R. Child et al. "Generating long sequences with sparse transformers". In: *arXiv:1904.10509* (2019).
[16] S. Jain et al. "A Heterogeneous and Programmable Compute-In-Memory Accelerator Architecture for Analog-AI using Dense 2D Mesh". In: *IEEE Trans. VLSI Systems* 31.1 (2023), pp. 114–127.
[17] P. Narayanan et al. "Toward on-chip acceleration of the backpropagation algorithm using nonvolatile memory". In: *IBM J. Res. Dev.* 61.4/5 (2017), 11:1–11:11.
[18] M. J. Rasch et al. "Hardware-aware training for large-scale and diverse deep learning inference workloads using in-memory computing-based accelerators". In: *arXiv:2302.08469* (2023).
[19] S. Ambrogio et al. "An Analog-AI Chip for Energy-Efficient Speech Recognition and Transcription". In: *Nature* (2023). to appear.
[20] V. Chan et al. "Yield Methodology and Learning in Phase Change Memory (PCM) technology for Analog Computing: Topic/category: YE: Yield Enhancement/Learning, YM: Yield Methodologies". In: *ASMC*. 2022, pp. 1–4.
[21] M. Le Gallo et al. "A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference". In: *Nature Electronics* (2023). to appear.
[22] A. Agrawal et al. "9.1 A 7nm 4-Core AI Chip with 25.6TFLOPS Hybrid FP8 Training, 102.4TOPS INT4 Inference and Workload-Aware Throttling". In: *ISSCC*. Vol. 64. 2021, pp. 144–146.
[23] H. Jia et al. "Scalable and programmable neural network inference accelerator based on in-memory computing". In: *IEEE J. Solid-State Circ.* 57.1 (2021), pp. 198–211.

Authorized licensed use limited to: Princeton University. Downloaded on October 12,2026 at 21:42:06 UTC from IEEE Xplore. Restrictions apply.